مدرسہ ریاضیات

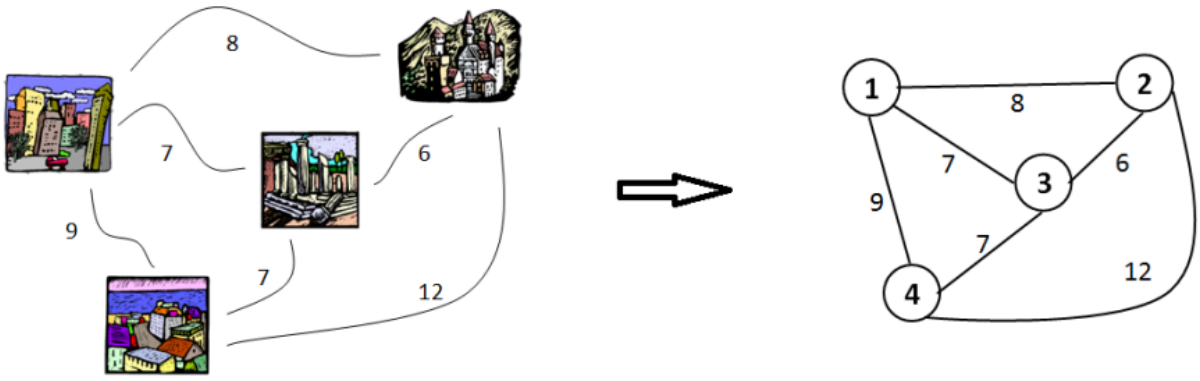
Lecture 05 – The Travelling Salesperson

Habib Math Club

1 Become the Travelling Salesperson!

You just moved into Turing Town and take up a job as a delivery driver for Lovelace and Co. You must drop off packages at 6 different houses every day and return home. Since you are a student and must save money, you track your fuel consumption. The first week, you try out different random routes and notice the different fuel costs. After a few weeks, you try to figure out what's the shortest possible route that visits each house exactly once and returns to your starting point?

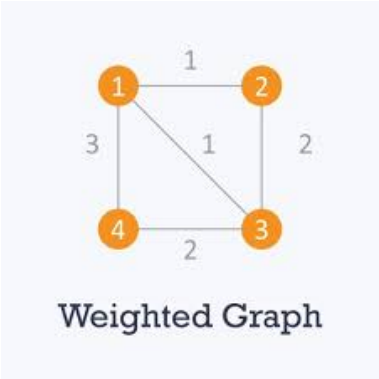
This challenge is called the **Travelling Salesperson Problem (TSP)**, a well-known and unsolved problem in computer science and mathematics.



2 Graph Representation

TSP is represented using a **weighted graph**, where:

- **Vertices (Nodes)** represent the cities/houses.
- **Edges** represent roads connecting the cities.
- **Weights** on edges denote distances or costs.

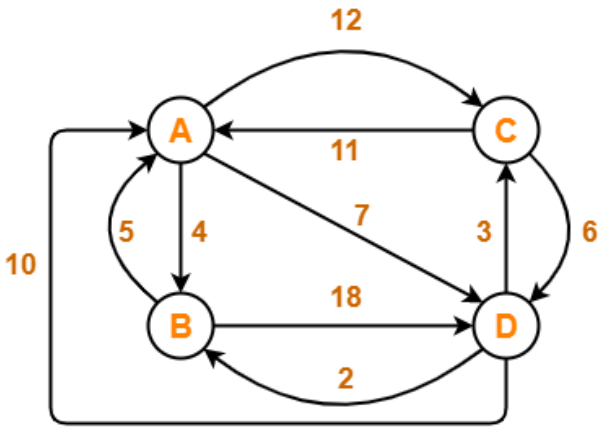


Hamiltonian Path: A path that visits every vertex exactly once.

Hamiltonian Cycle: A Hamiltonian Path that returns to the starting vertex or the shortest cycle that visits every node (house) exactly once and returns to the start.

3 Complexity of TSP

Consider a small delivery map with 4 locations: A, B, C, D. The weights represent the fuel cost from one city to another.



- A valid TSP route starts and ends at the same point.
- It must visit every other location exactly once.
- For 4 points, the total number of unique tours is:

$$(4-1)! = 3! = 6$$

Even for this small example, trying all possible tours is feasible, but the number of paths grows **factorially** with the number of cities.

4 Brute Forcing Our Way?

Suppose we have a fast computer that can check 1,000,000 routes per second. We can estimate computation time for N locations using:

$$(N-1)! \text{ routes to check.}$$

N	Time to compute all routes
6	Instantaneous
10	0.3 seconds
11	4 seconds
12	40 seconds
13	8 minutes
14	2 hours
15	1 day
25	$\sim$2 million years

Brute force becomes impractical very quickly due to exponential (factorial) growth, and even computers take a long time to solve this problem. So much so we have an entire field in computer science relating to it!

5 Common TSP Strategies and Algorithms

Apart from the brute-force solution, other strategies include:

- **Nearest Neighbor:** Choose the closest unvisited city at every step. It is fast, but doesn't always yield the best path.
- **Greedy Algorithm:** Pick the shortest available edge that doesn't lead to a cycle or revisit a city.

  There are other faster algorithms such as Dynamic and Linear Programming, which can reduce the time complexity of our algorithm.

6 Easy and Hard Problems in Computer Science

As the name suggests, there are two kinds of problems in Computer Science:

Easy Problems:

- Can be solved quickly (polynomial time, i.e the time required for a computer to solve a problem, where this time is a simple polynomial function of the size of the input).
- Examples: Sorting a list, finding the maximum number, simple search.

Hard Problems:

- Require enormous time to solve as input grows.

- Examples: Travelling Salesperson Problem (TSP), code breaking, optimal packing, Sudoku.

Some easy problems like matrix multiplication or prime number checking can still require advanced methods. Some hard problems can sometimes be approximated or partially solved efficiently, but exact solutions are ineffective for large inputs.

7 P vs NP and Complexity Theory

What even is P and NP?

- **P:** Problems that can be solved in **P**olynomial time.
- **NP:** Problems whose solutions can be verified in polynomial time, also called **N**on deterministic **P**olynomial time, even if finding that solution might be very hard. For example, Sudoku; solving a large puzzle might be hard, but once someone gives you a solution, it's easy to check that its correct. That's an NP problem.

The actual question is: Can P ever be equal to NP?

$$\boxed{\mathbf{P} \stackrel{?}{=} \mathbf{NP}}$$

If $P = NP$, it means all hard problems like TSP, Sudoku, protein folding, etc., can be solved efficiently. **Till now, no one knows the answer to this!** It's one of the biggest open questions in mathematics.

8 What Makes TSP Hard?

TSP belongs to a class of problems called **NP-hard**, which means:

- No known algorithm can solve all instances quickly, or in polynomial time.
- Even verifying the best possible answer is hard.
- Small increases in input size lead to exponential growth in computation.

TSP is also NP complete (both NP and NP Hard). **NP Complete** problems, like TSP and Sudoku, are the hardest problems in NP. If any one of them can be solved quickly, all NP problems can. **NP Hard** problems are at least as hard as NP problems, but might not even be checkable efficiently.
If someone gives you a route, it's easy to compute its total distance and check if it's under a given limit. But is TSP also P? Is there a fast (polynomial time) method to always find the shortest route?

In other words, if we generilse it, can we say: If it is easy to verify a solution (NP), is it also easy to find it (P)? If the answer is yes, i.e. P = NP, many hard problems, from scheduling deliveries to curing cancer, could suddenly become solvable.

To summarise:

$$P \subseteq NP \subseteq NP - Hard \subseteq Unsolvable$$

9 Real-World Applications

TSP-like problems appear in:

- Logistics and Delivery (e.g., Amazon, UPS)
- Ride-sharing routes (e.g., Uber, Careem)
- Circuit design (wiring shortest connections)
- Robotics path planning
- Astronomical scheduling (e.g., James Webb Space Telescope)

  "Simplicity is the final achievement. After one has played a vast quantity of notes and more notes, it is simplicity that emerges as the crowning reward of art." -Frédéric Chopin.